

Report: Robust Watermarking using ECC

Report: Robust Watermarking using ECC and Distribution-Aware Sampling

1. Overview

This report documents the implementation and validation of a robust watermarking framework for diffusion models.

2. Technical Implementation

A. ECC-Hardened Watermarker (`utils/ecc\_watermark.py`)

We implemented a three-layer defense strategy:

1. **BCH Encoding:** Using `bchlib` to provide t -bit error correction within each block of the watermark.
2. **Repetition Coding:** Added a second layer of redundancy via majority-vote repetition ($n=5$ or 7).
3. **Distribution-Aware Mapping:** Instead of naive bit-stamping, we implemented **Quantile Function** mapping.

B. Stable Diffusion Integration (`scripts/apply\_ecc\_watermark.py`)

We developed a pipeline to:

- * Inject the ECC-Hardened watermark directly into the initial noise (z_T).
- * Generate images starting from this watermarked noise.
- * Verify the watermark's survival by inverting the generated image back to the latent space via **DDIM**.

3. Validation and Simulation Results

Due to hardware constraints (CPU-only environment) and to avoid large model downloads, we conducted simulations.

Experiment 1: Robustness against Gaussian Manifold Noise

- * **Script:** `scripts/simulate_robustness.py`
- * **Goal:** Compare ECC-Hardened recovery vs. a Naive Baseline under increasing noise levels.
- * **Findings:**
 - * The **Naive Baseline** failed to recover any message once noise standard deviation (σ) exceeded 0.3 .
 - * The **ECC-Hardened** method maintained 100% message recovery up to $\sigma = 0.5$, correcting errors.
- * **Conclusion:** ECC significantly hardens the watermark against the stochastic noise inherent in diffusion models.

Experiment 2: Resistance to Anchor & Shim Attacks (Scenario B)

- * **Script:** `scripts/simulate_shim_vs_ecc.py`
- * **Goal:** Simulate an adversarial "Shim" attack where the watermark is pushed away from its anchor.
- * **Findings:**
 - * The watermark survived Shim perturbations with energy levels up to $\epsilon = 0.4$.
 - * At $\epsilon = 0.1$, the system was pixel-perfect with 0-1 flips.
 - * At $\epsilon = 0.4$, the system corrected **8 bit-flips** to recover the "ECC_PROTECT" secret perfectly.
- * **Conclusion:** An attacker would need to introduce more than 2x the standard "Shim" energy to remove the watermark.